DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2026)

Actividad Formativa 3

Computador básico

Requisitos

- Esta actividad es **formativa**: **no es evaluada** y **no asigna puntaje** a la nota del curso.
- El uso de navegador, equipos electrónicos distintos al entregado por el equipo docente, apuntes externos o cualquier programa distinto a Vivado está **prohibido**. En caso de infracción o sospecha de incumplimiento de este requisito se **expulsará** al estudiante del laboratorio.
- Esta actividad es de carácter **individual** y **presencial**.
- Su desarrollo es en **laboratorio** y deberá ser terminada dentro del horario.
- Deberá ser realizada en VHDL.

Objetivos

- Implementar registros de control.
- Implementar la lógica básica de la unidad de control.
- Construir un computador con la capacidad de hacer operaciones básicas.

Materiales Entregados

- Computador con Vivado WebPack instalado.
- Proyecto base de Vivado, con las señales definidas y *constraints* configurados.
- Una placa Basys3.

Criterios de logro (puntaje hipotético)

Importante: esta actividad **no es evaluada** y **no contribuye** a la calificación del curso. La escala siguiente es **solo referencial** (puntaje hipotético): sirve para orientar el nivel de avance esperado y para que el equipo docente comente el logro durante la verificación en placa. No se registra ni se suma a ninguna nota.
La verificación se realiza en la placa Basys3 ejecutando los tests otorgados en la actividad. Se podrá retirar tan pronto como una persona del equipo docente lo indique.

- **0,9 puntos hipotéticos (logro completo):** el componente **CPU** es construido correctamente y da el resultado *AA11* en todos los tests.
- **0,6 puntos hipotéticos (logro parcial):** el componente **CPU** es construido con algún fallo y da el resultado *AA11* en todos los tests menos uno.
- **0,3 puntos hipotéticos (logro parcial):** el componente **CPU** es construido con algún fallo y da el resultado *AA11* en un solo test.
- **0 puntos hipotéticos:** en cualquier otro caso (no se logra programar la placa, o la descripción no usa entidades, instancias y componentes como se requiere).

Hardware

Su trabajo será exclusivamente dentro de la CPU.

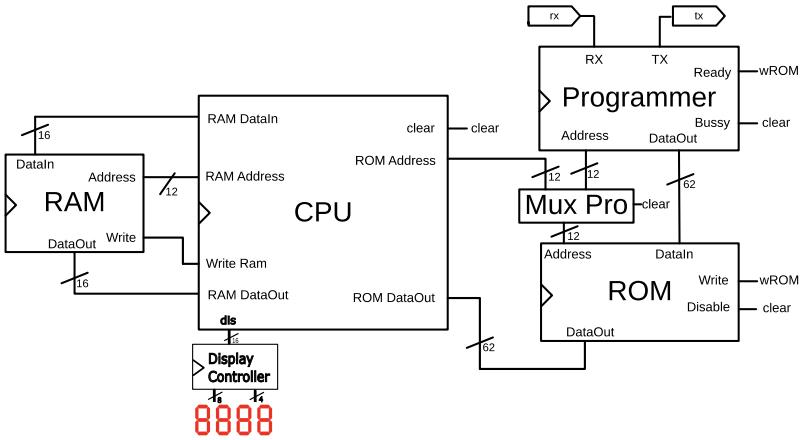


Figura 1: Diagrama parcial del computador básico de proyecto dentro del componente Basys3.

A diferencia del computador básico visto en clase, los registros, ALU y palabras de la memoria de datos (RAM) son de 16 bits, las direcciones de la RAM y la ROM son de 12 bits y la ALU no tiene soporte para operar sobre números en complemento de 2, es decir, tanto los operandos como el resultado se interpretan como números positivos en todo momento.

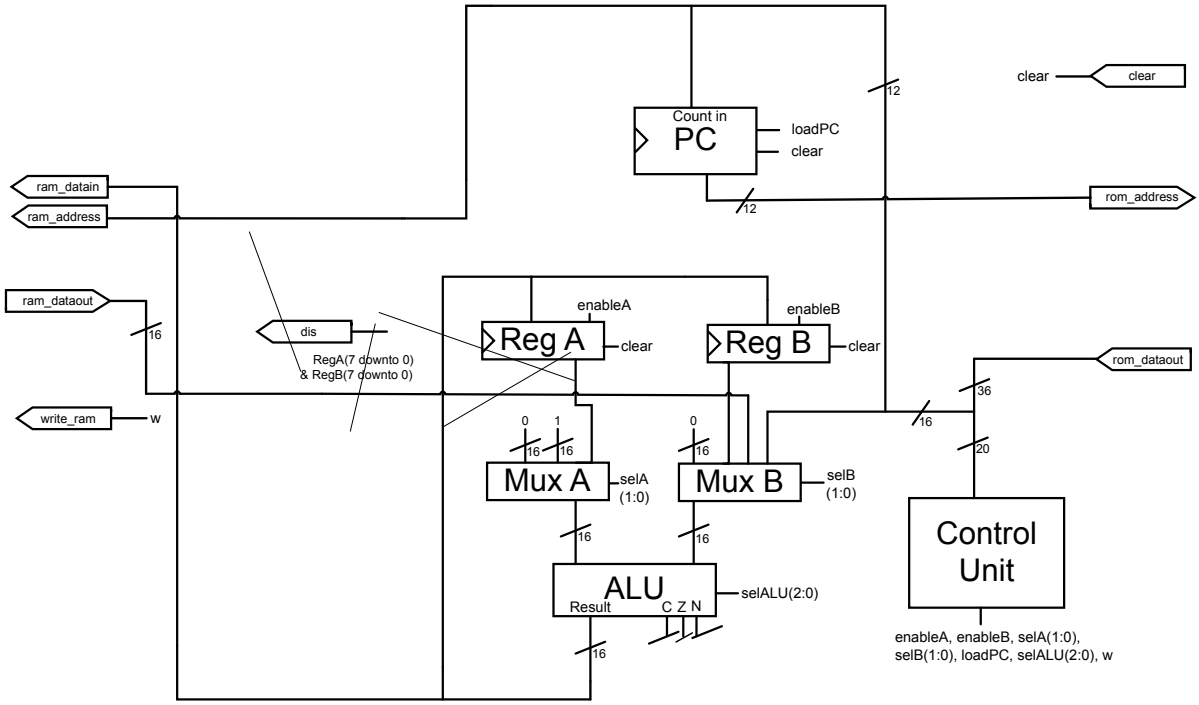


Figura 2: Diagrama interno de la CPU reducida del computador básico para la actividad.

Tareas

Su actividad **debe** tener lo siguiente:

- Dos **registros** `Reg A` y `Reg B` de uso general, de 16 *bits* cada uno (ya implementado como `Reg.vhd`, solo lo debe instanciar).
- Una **unidad aritmética lógica** `ALU` con 8 operaciones de 16 *bits* (ya implementado como `ALU.vhd`, solo lo debe instanciar).
- Un **registro contador** `PC` para el *address* de la instrucción del programa, de al menos 12 *bits* (ya implementado como `Reg.vhd`, solo lo debe instanciar).
- Dos **multiplexores** `Mux A` y `Mux B`, cada uno de cuatro entradas de 16 *bits*.
- Una **unidad de control** `Control Unit` implementada con **lógica combinacional** (ya semi-implentada, solo se debe completar su lógica interna).
- Usar ensamblador para probar arquitectura.

Hint: No es necesario crear diferentes componentes para cada una de ellas, recuerden que en Vivado pueden crear distintas **instancias** de una componente para los elementos que tienen el mismo comportamiento.

Software

Se utilizará un ensamblador para traducir y transferir distintos programas en lenguaje de *Assembly* a la `ROM` de su computador y, de ese modo, poder ejecutarlos. Escriba los programas que estime convenientes para probar su arquitectura.

En la siguiente tabla, se describe cómo las palabras en la ROM son codificadas para representar las instrucciones. El ensamblador entregado se encargará de leer las instrucciones en *Assembly*, codificarlas y enviarlas al computador programable. En caso de que ninguno de los *bits* de la palabra de la ROM esté activo para la selección de un multiplexor, estos deben seleccionar una de sus señales de entrada de forma arbitraria a través de `with others` en Vivado.

Bits	**Acción**
1 al 0	00: En el MuxA se selecciona el Registro A 10: En el MuxA se selecciona el valor Uno Otros: En el MuxA se selecciona el valor Cero
3 al 2	00: En el MuxB se selecciona el Registro B 01: En el MuxB se selecciona el Literal 10: En el MuxB se selecciona la salida de la RAM 11: En el MuxB se selecciona el valor Cero
6 al 4	01: Selección de la operación de la ALU
7	1 cuando se habilita la carga en el registro A
8	1 cuando se habilita la carga en el registro B
9	1 cuando se habilita la escritura en la RAM
19 al 10	**No asignados en esta actividad**

Tabla 1: Codificación de la palabra del computador básico programable.

Pruebas

Para validar el correcto funcionamiento de su arquitectura, se probará con archivos `.exe` que contienen código que será procesado por el ensamblador. Es esperado que, tras la correcta ejecución de cada programa de prueba en su CPU, el resultado final que termine visualizándose en los *displays* de 7 segmentos de la placa sea `AA11`.

Assembly

Actividad Formativa 3

MOV	A,B	Guarda el valor de B en A
	B,A	Guarda el valor de A en B
	A,Lit	Guarda un literal Lit en A
	B,Lit	Guarda un literal Lit en B
	A,(Dir)	Guarda el valor de Mem[Dir] en A
	B,(Dir)	Guarda el valor de Mem[Dir] en B
	(Dir),A	Guarda el valor de A en Mem[Dir]
	(Dir),B	Guarda el valor de B en Mem[Dir]
ADD SUB AND OR XOR	A,B	Guarda el resultado de A op B[1] en A
	B,A	Guarda el resultado de A op B en B
	A,Lit	Guarda el resultado de A op Lit en A
	B,Lit	Guarda el resultado de A op Lit en B
	A,(Dir)	Guarda el resultado de A op Mem[Dir] en A
	B,(Dir)	Guarda el resultado de A op Mem[Dir] en B
	(Dir)	Guarda el resultado de A op B en Mem[Dir]
NOT SHL SHR	A	Guarda el resultado de op A en A
	B,A	Guarda el resultado de op A en B
	(Dir),A	Guarda el resultado de op A en Mem[Dir]
INC	A	Incrementa el valor de A en una unidad
	B	Incrementa el valor de B en una unidad
	(Dir)	Incrementa el valor de Mem[Dir] en una unidad
DEC	A	Decrementa el valor de A en una unidad
NOP		No hace cambios